

Assignment 6

Concurrency Control of Transactions Using Timestamp Ordering

Due Monday, May 5, 2025

This assignment extends the key-value store you developed for Assignment 4. As in that assignment, a single server receives RPC requests from multiple concurrent clients to perform read and write operations on a key-value store. This time, each client encapsulates its logically ordered sequence of read and write operations on the store within a transaction context, explicitly demarcated by `BeginTransaction()` to initiate the transaction and `Commit()` to finalize it.

Your task is to write a Go program `server.go` that maintains the key-value store and executes the clients' transactions according to the **Timestamp Ordering** concurrency control scheme discussed in class. (See Lecture 16, slide 46ff. and Section 16.6 of the textbook.) The `client.go` program will be provided to you; it is responsible for creating client transactions, which are then executed by the server through RPC calls.

For this assignment, we will assume that the server maintains a key-value store containing just two keys, `x` and `y` (both strings), each starting with a value of zero (their initial committed value). Transactions may only perform read and write operations on these two keys—there is no delete operation. A read or write operation on a key other than `x` or `y` will result in an error ("key not found"). Your `server.go` program should create the key-value store with just these two keys and their initial values before any transaction is processed.

The server program implements a transaction service, exporting to the client program the following methods: `BeginTransaction()`, `Read()`, `Write()`, and `Commit()`. The specifications of these methods are given below.

BeginTransaction()

```
type Timestamp int64

type BeginArgs struct{
    TxID string
}

type BeginReply struct {
    Timestamp Timestamp
}

func (s *TransactionService) BeginTransaction(args *BeginArgs, reply
*BeginReply) error
```

`BeginTransaction()` takes a transaction ID, `args.TxID`, which is a string (e.g., "Alice", "Bob", etc.) provided by the client that uniquely identifies the transaction. It should return in `reply.Timestamp` a unique timestamp that defines the transaction's position in the time

sequence of transactions. One way to achieve this is shown below; it returns the current time in microseconds since January 1, 1970 UTC. (Be sure to include the `time` package):

```
ts := Timestamp(time.Now().UnixMicro())
reply.Timestamp = ts
```

`BeginTransaction()` should then create an object (a `struct`) for this transaction that contains the transaction ID and its timestamp as fields. (You may add additional fields to this struct as needed.)

`Read()`

```
type ReadArgs struct{
    TxID  string
    Key   string
}

type ReadReply struct {
    Value int
    Error string
}

func (s *TransactionService) Read(args *ReadArgs, reply *ReadReply)
error
```

`Read()` takes a transaction ID, `args.TxID`, and a key `args.Key`, and reads the key's value from the store. It should return the value read in `reply.Value`. It should implement the **Read Rule** for Timestamp Ordering (See Lecture 16, slides 51-52 and pages 714-715 of the textbook.) If any of the following errors occur, it should write the indicated string in `reply.Error` and return.

- **"Transaction not found"**: the transaction `args.TxID` does not exist
- **"Key not found"**: the key `args.Key` does not exist in the key-value store
- **"Aborted: read timestamp <= committed write timestamp"**: the transaction's timestamp $\leq$ committed write timestamp on the key

Write()

```
type WriteArgs struct{
    TxID  string
    Key   string
    Value int
}

type WriteReply struct {
    Error string
}

func (s *TransactionService) Write(args *WriteArgs, reply *WriteReply)
error
```

Write() takes a transaction ID, `args.TxID`, a key `args.Key`, and the value of the key, `args.Value`, to be written to the store. It should implement the **Write Rule** for Timestamp Ordering (See Lecture 16, slides 49-50 and page 713 of the textbook.) If any of the following errors occur, it should write the indicated string in `reply.Error` and return.

- "Transaction not found": the transaction `args.TxID` does not exist
- "Key not found": the key `args.Key` does not exist in the key-value store
- "Aborted: write timestamp < max read timestamp": the transaction's timestamp < maximum read timestamp on the key
- "Aborted: write timestamp <= committed write timestamp": the transaction's timestamp $\leq$ committed write timestamp on the key

Commit()

```
type CommitArgs struct{
    TxID  string
}

type CommitReply struct {
    Success bool
    Error   string
}

func (s *TransactionService) Commit(args *CommitArgs, reply
*CommitReply) error
```

`Commit()` commits the transaction with ID `args.TxID`, applying its tentative writes (if any) to the store. It should return `reply.Success = true` if the commit is successful. If any of the following errors occur, it should write the indicated string in `reply.Error` and return.

- `"Transaction not found"`: the transaction `args.TxID` does not exist
- `"Commit failed: transaction has been aborted"`: the transaction was aborted during a previous read or write operation

The server's RPC replies inform the client of the results of the transaction operations, including errors, if any. In addition, the server must produce output that documents how the operations are performed. This is explained in the sample runs given later.

The Client Program

The client program `client.go` can be downloaded from the Assignments/Assignment 6 folder in the Files section of the Canvas site. It is also available on the `csprojects` server (copy it from `~palis/client.go`).

The client program communicates with the server on the same host machine via TCP port 1234. Therefore, you can run `server.go` and `client.go` on the same machine on two separate terminals. The client program takes a single command-line argument: an integer ≤ 7 indicating the number of client transactions created and sent to the server for processing. Each transaction is created by a separate goroutine that calls the function `transactionWorker()`, which issues the following RPCs to the server: (1) `BeginTransaction()`; (2) a random sequence of three `Read()` and `Write()` operations on keys `x` and `y`; and (3) `Commit()`. Each transaction is given a unique name as its transaction ID ("Alice", "Bob", "Charlie", "Dave", "Eve", "Frank", or "Grace"). For each operation, the program outputs the result if the operation is successful, or an error message if it fails.

Sample Run 1

A sample run of `client.go` for two transactions with IDs **Alice** and **Bob** is shown in Figure 1-(a). Because these transactions are launched as separate goroutines, the order in which these goroutines are executed depends on the Go runtime scheduler. Moreover, outstanding RPCs are multiplexed over a single TCP connection, and therefore, the RPCs are not necessarily sent to the server and their results received by the client in program order. The important thing to note is the **timestamps** assigned by the server to these transactions: Alice has a smaller timestamp than Bob (highlighted in **red**). Consequently, **Timestamp Ordering**, when correctly implemented, should ensure that Alice commits first before Bob.

Let's go through the rest of the output produced by the client program. On line 3, Bob wrote `y = 446`, yet on line 4 Alice read `y = 0`. This is correct because, per the Write Rule, Bob's write only results in a tentative write to `y`; it will be applied to the key-value store only when Bob commits. Per the Read Rule, Alice reads the current committed value of `y`, which is zero (its initial value). On lines

5 and 6, Alice wrote $x = 187$, then wrote $x = 166$. The first write results in a tentative write to x ; the second updates the value of this tentative write. On line 7, Alice's commit causes the tentative write to be applied to the store. Consequently, when Bob reads on line 8, it reads $x = 166$, which is the (new) committed value of x . Bob then performs a tentative write to x on line 9, which will be written to the store when it commits (line 10).

```
> go run client.go 2
1 [Bob] Began transaction with timestamp 1745292347528479
2 [Alice] Began transaction with timestamp 1745292347528322
3 [Bob] Wrote y = 446
4 [Alice] Read y = 0
5 [Alice] Wrote x = 187
6 [Alice] Wrote x = 166
7 [Alice] Commit succeeded
8 [Bob] Read x = 166
9 [Bob] Wrote x = 422
10 [Bob] Commit succeeded
```

Figure 1-(a)

```
> go run server.go
1 RPC server running on localhost:1234
2 [Alice] Began transaction with timestamp 1745292347528322
3 [Bob] Began transaction with timestamp 1745292347528479
4 [Bob] Attempting to write y = 446
5 [Bob] Tentatively write y = 446
6 [Alice] Attempting to read y
7 [Alice] Read y = 0
8 [Alice] Attempting to write x = 187
9 [Alice] Tentatively write x = 187
10 [Alice] Attempting to write x = 166
11 [Alice] Tentatively write x = 166
12 [Bob] Attempting to read x
13 [Alice] Attempting to commit
14 [Alice] Commit succeeded
15 [Alice] Commit value of y = 0
16 [Alice] Commit value of x = 166
17 [Bob] Read x = 166
18 [Bob] Attempting to write x = 422
19 [Bob] Tentatively write x = 422
20 [Bob] Attempting to commit
21 [Bob] Commit succeeded
22 [Bob] Commit value of x = 422
23 [Bob] Commit value of y = 446
```

Figure 1-(b)

Figure 1-(b) shows the corresponding run of the server program `server.go`. It illustrates the expected output from your server program, documenting how transaction operations are performed. In general, your server program should produce an appropriate output for each of the following (preface each output with the transaction ID):

- When it executes `BeginTransaction()`: If successful, output the timestamp assigned to the transaction. If not, output an error message.
- When it executes `Read()`: Output “Attempting to read <key>” at the start of the operation. If successful, output “Read <key> = <value>”. If not, output the cause of the error; if the transaction is aborted, output the reason it was aborted.
- When it executes `Write()`: Output “Attempting to write <key> = <value>” at the start of the operation. If a tentative write is performed, output “Tentatively write <key> = <value>”. If an error occurs, output the cause of the error; if the transaction is aborted, output the reason it was aborted.
- When it executes `Commit()`: Output “Attempting to commit” at the start of the operation. If successful, output “Commit succeeded” followed by the values of keys `x` and `y` upon commit. If not, output an error message.

As you can see, the server output describes in detail the steps it carried out when executing each transaction operation, which could only be inferred from the client output.

Sample Run 2

Figures 2-(a) and 2-(b) display the outputs of `client.go` and `server.go`, respectively, for another run involving two transactions with IDs **Alice** and **Bob**. Inspection of the client program’s output indicates that transaction Bob read `y = 0` (line 3). Transaction Alice then attempted to write `y = 226` but was aborted (line 4). This is correct because Bob has a higher timestamp than Alice and, per the Write Rule, Alice’s write came too late: the key `y` has already been read by another transaction (Bob) with a higher timestamp. The server program’s output confirms this: Alice attempted to write `y = 226` (line 6) but was aborted (line 7) when the server discovered that Alice’s timestamp is less than Bob’s timestamp. Furthermore, Bob’s attempt to write `y = 163` (line 12) results in a tentative write (line 13), which is applied to the store only when Bob commits (line 15).

```
> go run client.go 2
1 [Bob] Began transaction with timestamp 1745290860144791
2 [Alice] Began transaction with timestamp 1745290860144737
3 [Bob] Read y = 0
4 [Alice] Error on write y = 226: Aborted: write timestamp <
  max read timestamp
5 [Bob] Read y = 0
6 [Bob] Wrote y = 163
7 [Bob] Commit succeeded
```

Figure 2-(a)

```
> go run server.go
1  RPC server running on localhost:1234
2  [Alice] Began transaction with timestamp 1745290860144737
3  [Bob] Began transaction with timestamp 1745290860144791
4  [Bob] Attempting to read y
5  [Bob] Read y = 0
6  [Alice] Attempting to write y = 226
7  [Alice] Write y aborted: write timestamp < max read timestamp
8  [Alice] Attempting to abort
9  [Alice] Abort succeeded
10 [Bob] Attempting to read y
11 [Bob] Read y = 0
12 [Bob] Attempting to write y = 163
13 [Bob] Tentatively write y = 163
14 [Bob] Attempting to commit
15 [Bob] Commit succeeded
16 [Bob] Commit value of x = 0
17 [Bob] Commit value of y = 163
```

Figure 2-(b)

Aborting and Committing Transactions

In your server program, make sure to perform the following “cleanup” steps when aborting and committing a transaction.

- When aborting a transaction T , the server must remove all tentative writes posted by T —i.e., T 's entry must be removed from every key's Tentative Write (TW) list, if it exists. Similarly, T 's entry must be removed from every key's Read Timestamp (RTS) list, if it exists.
- Before committing a transaction T , the server should first check whether *all* transactions with timestamps less than T have either been committed or aborted. Then and only then can T be committed. Upon commit, all tentative writes posted by T must be applied to the key-value store; keys that have been rewritten by T must update their timestamps to T 's timestamp. Moreover, T 's entries in the TW lists of every key must be removed. However, T 's entries in the keys' RTS list should remain, as they will still be used to enforce timestamp ordering for later transactions.
- There are cases when a transaction must wait until another transaction is aborted or committed before it can proceed (e.g., during commit or when applying the Read Rule). One way to implement this is to associate a notification channel with each transaction. When a transaction commits or aborts, you close the channel or send a message to it. Other transactions can listen to this channel to get notified and act accordingly. See <https://gobyexample.com/channel-synchronization> for a simple example.

Collaboration Policy

For this assignment, you may work with one other student in the class, forming a team of two. Each team should submit a single assignment, and both team members will receive the same grade. If you decide to form a team with another student, you and your teammate should designate one person to serve as the primary contact for the instructor and to submit the assignment on behalf of your team. The primary contact for your team must notify the instructor of your team members by midnight on Thursday, April 24.

If you choose to work alone for this assignment, no action is required. If the instructor has not received notice of your team membership by April 24, it will be assumed that you are completing and submitting the assignment individually.

Submitting your Assignment

- Your Go source code should be well-structured and thoroughly commented to enhance readability and maintainability. At the top of each source file, include a comment with the file name, your name and net ID (if working individually), or the names and net IDs of members of your team.
- Create a tar archive containing your source files and upload the archive to the Canvas course site.
- This assignment is due at 11:59 p.m. on Monday, May 5. No late assignments will be accepted.

